

To create controller in getx:

```
import 'package:flutter_application_8/app/modules/product/models/product.dart';
import 'package:get/get.dart';
```

```
class ProductController extends GetxController {
  var products = <ProductModel>[].obs;
}
```

In this way <ProductModel>[].obs we create an observable for list of ProductModel-Class.

Like stateful widget, getx controller has OnInit-Method:

```
class ProductController extends GetxController {
  var products = <ProductModel>[].obs;

  @override
  void onInit() {
    products.addAll([
      ProductModel(name: "JLoka Product-01", price: 150.5),
      ProductModel(name: "JLoka Product-02", price: 200.5),
      ProductModel(name: "JLoka Product-02", price: 300.5),
    ]);

    super.onInit();
  }
}
```

Getx manage the dependency injection for our view, so to add product controller for product view:

```
import 'package:get/get.dart';

class ProductView extends StatelessWidget {
  final ProductController productController = Get.put(ProductController());

  ProductView({super.key});

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
*****
```

To use controller && its observer objects inside the body of scaffold we must use Obx from get package.

To make route depending on name, we can use Get.toNamed.

```
import 'package:get/get.dart';

class ProductView extends StatelessWidget {
  final ProductController productController = Get.put(ProductController());

  ProductView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Product List"), centerTitle: true),

      body: Obx(() {
        return ListView.builder(
          itemCount: productController.products.length,
          itemBuilder: (BuildContext context, int index) {
            final product = productController.products[index];

            return ListTile(
              title: Text(product.name),
              subtitle: Text("\${product.price}"),
              onTap: () {
                Get.toNamed("/product-detailed", arguments: product);
              },
            );
          },
        );
      });
  }
}
```

```

        },
      );
    }},
  );
}
}
*****

```

To define the pages with routes:

```

// ignore_for_file: constant_identifier_names

part of 'app_pages.dart';

class Routes {
  static const PRODUCT = "/product";
  static const PRODUCT_DETAIL = "/product-detail";
}

```

Then we define the pages using GetPage:

```

import 'package:get/get.dart';

part 'app_routes.dart';

class AppPages {
  static final routes = [
    GetPage(name: Routes.PRODUCT, page: () => ProductView()),
    GetPage(name: Routes.PRODUCT_DETAIL, page: () => ProductDetailView()),
  ];
}

*****

```

To use the arguments of toNamed:

```
class ProductDetailView extends StatelessWidget {
  ProductDetailView({super.key});

  final ProductModel product = Get.arguments;

  ....

}
```

To use the getx controller, routes, observables, ...etc, we must use GetMaterialApp not MaterialApp, and we can change the main app to stateless widget:

```
import 'package:flutter/material.dart';
import 'package:flutter/widgets.dart';
import
'package:flutter_application_8/app/modules/product/controllers/cart_controller.dart';
import 'package:get/get.dart';

class CartView extends StatelessWidget {
  CartView({super.key});

  final CartController cartController = Get.find<CartController>();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Cart View"), centerTitle: true),

      body: Column(
        children: <Widget>[
          Expanded(
            child: Obx(() {
              return ListView.builder(
                itemCount: cartController.cartItems.length,
                itemBuilder: (BuildContext context, int index) {
                  final cartItem = cartController.cartItems[index];
                  return ListTile(
```

```

        title: Text(cartItem.name),
        subtitle: Text("\${cartItem.price}"),
        trailing: IconButton(
            onPressed: () => cartController.removeFromCart(cartItem),
            icon: Icon(Icons.remove_circle_outline),
        ),
    );
},
);
}),
),
),

Obx(() {
    return Padding(
        padding: EdgeInsets.all(16.0),
        child: Text(
            "Total: ${cartController.totalAmount.value.toStringAsFixed(2)}",
            style: TextStyle(fontSize: 24.0, color: Colors.blueGrey),
        ),
    );
}),
],
),
);
}
}
}
*****

```

Note 1: To use the controller that we want to use in the app, we must call **put-method**, before calling **find-method**.

Note 2: To set controller for all pages, we can set it before calling **runApp**

```

*****

void main() {
    Get.put(CartController());
    runApp(const MyApp());
}
*****

```

Reactive variable, when it changed, the UI will change automatically.

So, when we call **addToCart**, And **removeFromCart** the observer-variables will change so the main ui will change also.

Note: if we use Obx without reactive variables, we will get error.

To remove all the screens, and navigate to destination:

```
ElevatedButton(  
  onPressed: () {  
    Get.offAllNamed("/product");  
  },  
  child: Text("Go to product"),  
),  
*****
```

Types of navigation in getx:

- Get.off
- Get.offAll
- Get.back
- Get.to
- Get.offAllNamed

For get design, we must separate the binding of controllers from views pages.

In this way we create binds for controller/s in getx:

Note: when we use this bindings it will initialize of controller (ProductController).

```
import
'package:flutter_application_8/app/modules/product/controllers/product_controller
.dart';
import 'package:get/get.dart';

class ProductBindings extends Bindings {
  @override
  void dependencies() {
    Get.put(ProductController());
  }
}
*****
```

To use bindings with views, we set it with bindings of GetPage

```
GetPage(
  name: Routes.PRODUCT,
  page: () => ProductView(),
  binding: ProductBindings(),
),
*****
```

After that to use the controller, we use find inside ProductView:

```
final ProductController productController = Get.find<ProductController>();
*****
```

The lazyPut of getx only initialize the controller, when the find method is called.

The problem of lazyPut, is that it will delete the controller from memory when we exit from page.

So, when the controller is deleted it will not accessible to views pages.

To solve the above problem, we must:

1. Create separate binds for CartController
2. Use lazyPut for that controller, inside the bindings
3. Use the prev. binding inside the GetPage of CartDetailsView.

The main benefit of lazyPut it will handle the memory of app.

```
class AppPages {
  static final routes = [
    GetPage(
      name: Routes.PRODUCT,
      page: () => ProductView(),
      binding: ProductBindings(),
    ),
    GetPage(
      name: Routes.PRODUCT_DETAIL,
      page: () => ProductDetailView(),
      binding: CartBinding(),
    ),
    GetPage(name: Routes.CART, page: () => CartView(), binding: CartBinding()),
    GetPage(name: Routes.CHECKOUT, page: () => CheckoutView()),
  ];
}
```

When we create application that depends on Controller between pages, we must init the controller inside the main.dart, else if we use lazyPut it will clear the data, when we close the page or go to any other page.

In this way we can customize the getx snackbar:

```
Get.snackbar(  
    "Add to cart",  
    "${product.name} added to your cart",  
    snackPosition: SnackPosition.BOTTOM,  
    duration: Duration(milliseconds: 2000),  
    backgroundColor: Colors.blueGrey,  
    colorText: Colors.white,  
    showProgressIndicator: true,  
);  
*****  
  
Get.defaultDialog(  
    title: "Remove Item",  
    middleText:  
        "Are you sure to remove ${cartItem.name}?",  
    textConfirm: "Ok",  
    textCancel: "cancel",  
    confirmTextColor: Colors.white,  
    onConfirm: () {  
        cartController.removeFromCart(cartItem);  
        Get.back(); // to pop up the dialog from screen  
    },  
    onCancel: () {},  
);  
*****
```

To use the GetStorage:

```
import 'package:get/get.dart';
import 'package:get_storage/get_storage.dart';

class CartService extends GetxService {
  final cartStorage = GetStorage();

  var cartItems = <ProductModel>[].obs;

  @override
  void onInit() {
    super.onInit();

    List? storedCart = cartStorage.read("cartItems");

    if (storedCart != null) {
      cartItems.assignAll(
        storedCart.map((e) => ProductModel.fromJson(e)).toList(),
      );
    }

    ever(cartItems, (_) {
      cartStorage.write("cartItems", cartItems.map((e) => e.toJson()).toList());
    });
  }

  void addToCart(ProductModel product) {
    cartItems.add(product);
  }

  void removeFromCart(ProductModel product) {
    cartItems.remove(product);
  }

  void clearCart() {
    cartItems.clear();
  }
}
```

Then to init the GetStorage:

```
void main() async {  
  Get.put(CartController());  
  await GetStorage.init();  
  runApp(const MyApp());  
}  
*****
```